

""

=====

=====

AFZAL AI BOT

Scalping Signal Bot (Crypto + Forex) ->
Telegram

1-Min + 3-Min Multi-Timeframe, 5-Indicator
Confirmation

=====

=====

Kya karta hai:

1. Binance se Crypto candles leta hai
(1min aur 3min dono)

2. Twelve Data se Forex candles leta hai
(1min aur 3min dono)

3. 5 indicators calculate karta hai:

- RSI
- MACD
- EMA crossover (fast/slow)
- Bollinger Bands
- Stochastic Oscillator

4. Har timeframe pe kitne indicators
BUY/SELL ke haq mein hain, unka

score nikalta hai. Signal tabhi bhejta
hai jab:

- 1min AUR 3min dono timeframe same
direction confirm karein

- Kam se kam MIN_CONFIRMATIONS
indicators agree karein

5. Telegram Bot API se signal message
bhejta hai

6. Same signal dobara turant nahi bhejta
(state file mein save hota hai)

=====

=====

ZAROORI DISCLAIMER (please read):

- Ye bot sirf technical-indicator based
signals deta hai.

- Ye financial advice NAHI hai. Trading
(especially 1-3 min scalping
ya binary-style trading) mein bohat risk
hota hai.

- OTC / synthetic pairs pe koi bhi
technical indicator meaningful
nahi hota kyunki wahan price broker khud
generate karta hai. Ye

bot sirf REAL live market data (Binance
crypto + real Forex) pe
kaam karta hai.

- Pehle demo/paper account pe test karein,
phir hi real paise lagayein.

=====

=====

"""

import json

import os

import time

import requests

import pandas as pd

```

-----
# 1) CONFIGURATION -- apni details yahan
bharein
# -----
-----

CONFIG = {
    # --- Telegram Bot settings ---
    "TELEGRAM_BOT_TOKEN":
"8260776993:AAF9RShKUj3sKK_shooapH-Eg-
D4MIpWndU",
    "TELEGRAM_CHAT_ID": "8683932044",

    # --- Twelve Data (Forex) API key ---
    "TWELVE_DATA_API_KEY":
"fc1a63a91270492fbb1f3170037287f5",

    # --- Symbols to watch (sab major
pairs) ---
    "CRYPTO_SYMBOLS": ["BTC", "ETH", "LTC",
"DOGE"], # display names
    "FOREX_PAIRS": [
        "EUR/USD", "GBP/USD", "USD/JPY",
"USD/CHF",
        "AUD/USD", "USD/CAD", "NZD/USD",
"EUR/JPY",
        "GBP/JPY", "EUR/GBP", "EUR/CHF",
"AUD/JPY",
        "USD/SGD", "EUR/AUD", "GBP/CHF",
    ], # Twelve Data format -- Quotex
live forex majors+minors se match

    # --- Timeframes for scalping (1min +
3min confirmation) ---

```

```
"CRYPTO_INTERVALS": ["1m", "3m"],      #
3m Kraken se resample hota hai 1m se
"FOREX_INTERVALS": ["1min", "3min"],  #
Twelve Data format

# Kraken (crypto data source) ke apne
pair codes -- Binance US se block ho gaya
tha
"CRYPTO_KRAKEN_MAP": {
    "BTC": "XBTUSD",
    "ETH": "ETHUSD",
    "LTC": "LTCUSD",
    "DOGE": "DOGEUSD",
},

# --- Indicator settings ---
"RSI_PERIOD": 14,
"RSI_OVERSOLD": 30,
"RSI_OVERBOUGHT": 70,

"MACD_FAST": 12,
"MACD_SLOW": 26,
"MACD_SIGNAL": 9,

"EMA_FAST": 9,
"EMA_SLOW": 21,

"BB_PERIOD": 20,
"BB_STD": 2,

"STOCH_PERIOD": 14,
"STOCH_SMOOTH": 3,
"STOCH_OVERSOLD": 20,
"STOCH_OVERBOUGHT": 80,
```

```

        # Kam se kam kitne (5 mein se)
        indicators agree karein tabhi signal maana
        jaye
        "MIN_CONFIRMATIONS": 3,

        # --- Twelve Data free plan rate limit
        safety ---
        # Free plan = 8 requests/minute. Isse
        zyada tez bhejne se error aata hai.

        "TWELVE_DATA_MIN_SECONDS_BETWEEN_CALLS": 8,

        # --- How often bot checks the market
        (seconds) ---
        "CHECK_INTERVAL_SECONDS": 60,    # 60 =
        1 minute

        "STATE_FILE": "last_signals.json",
        "SIGNAL_COOLDOWN_SECONDS": 180, # same
        pair pe dobara signal se pehle kam se kam
        itna wait
    }

    BOT_NAME = "Afzal AI Bot"

    # -----
    -----
    # 2) DATA FETCHING (OHLC --
    open/high/low/close, indicators ke liye)
    # -----
    -----

```

```

def get_crypto_candles_raw_1m(kraken_pair,
limit=200):
    """Kraken public API se 1-min OHLC
    candles leta hai (globally accessible, koi
    region-block nahi)."""
    url =
    "https://api.kraken.com/0/public/OHLC"
    params = {"pair": kraken_pair,
    "interval": 1}
    resp = requests.get(url, params=params,
    timeout=15)
    resp.raise_for_status()
    data = resp.json()
    if data.get("error"):
        raise ValueError(f"Kraken error for
    {kraken_pair}: {data['error']}")
    result = data.get("result", {})
    series_key = next((k for k in
    result.keys() if k != "last"), None)
    if series_key is None:
        raise ValueError(f"Kraken: no data
    returned for {kraken_pair}")
    rows = result[series_key]
    df = pd.DataFrame(rows, columns=
    ["time", "open", "high", "low", "close",
    "vwap", "volume", "count"])
    df = df.astype({"high": "float64",
    "low": "float64", "close": "float64"})
    return df[["high", "low",
    "close"]].tail(limit).reset_index(drop=True
    )

def resample_ohlc(df, factor):

```

```

    """1-min candles ko group karke bade
    timeframe (jaise 3-min) mein badalta
    hai."""
    n = len(df) // factor
    if n == 0:
        return df.iloc[0:0].copy()
    rows = []
    for i in range(n):
        chunk = df.iloc[i * factor: (i + 1)
* factor]
        rows.append({
            "high": chunk["high"].max(),
            "low": chunk["low"].min(),
            "close":
chunk["close"].iloc[-1],
        })
    return pd.DataFrame(rows)

def
get_crypto_multi_timeframe(display_symbol,
cfg):
    """Ek hi Kraken call se 1-min aur 3-min
    (resampled) dono candles return karta
    hai."""
    kraken_pair =
cfg["CRYPTO_KRAKEN_MAP"].get(display_symbol
)
    if not kraken_pair:
        raise ValueError(f"Kraken mapping
nahi mili: {display_symbol}")
    df_1m =
get_crypto_candles_raw_1m(kraken_pair)
    df_3m = resample_ohlc(df_1m, 3)

```

```

        return {"1m": df_1m, "3m": df_3m}

_last_twelve_data_call_time = [0.0]

def _throttle_twelve_data(cfg):
    """Twelve Data free plan (8 req/min) se
    zyada tez requests na jayein, isliye wait
    karta hai."""
    min_gap =
cfg["TWELVE_DATA_MIN_SECONDS_BETWEEN_CALLS"
]
    elapsed = time.time() -
_last_twelve_data_call_time[0]
    if elapsed < min_gap:
        time.sleep(min_gap - elapsed)
        _last_twelve_data_call_time[0] =
time.time()

def get_forex_candles(pair, interval,
api_key, output_size=100, cfg=None):
    """Twelve Data API se OHLC forex candle
    data leta hai (rate-limit safe)."""
    if cfg is not None:
        _throttle_twelve_data(cfg)
    url =
"https://api.twelvedata.com/time_series"
    params = {
        "symbol": pair,
        "interval": interval,
        "outputsize": output_size,
        "apikey": api_key,

```

```

    }
    resp = requests.get(url, params=params,
timeout=15)
    resp.raise_for_status()
    data = resp.json()
    if "values" not in data:
        raise ValueError(f"Twelve Data
error for {pair}: {data}")
    values = list(reversed(data["values"]))
    df = pd.DataFrame({
        "high": [float(v["high"]) for v in
values],
        "low": [float(v["low"]) for v in
values],
        "close": [float(v["close"]) for v
in values],
    })
    return df

# -----
-----
# 3) INDICATORS
# -----
-----

def calculate_rsi(close, period=14):
    delta = close.diff()
    gain = delta.clip(lower=0)
    loss = -delta.clip(upper=0)
    avg_gain =
gain.rolling(window=period).mean()
    avg_loss =
loss.rolling(window=period).mean()

```

```

    rs = avg_gain / avg_loss.replace(0, 1e-
10)
    return 100 - (100 / (1 + rs))

def calculate_macd(close, fast=12, slow=26,
signal=9):
    ema_fast = close.ewm(span=fast,
adjust=False).mean()
    ema_slow = close.ewm(span=slow,
adjust=False).mean()
    macd_line = ema_fast - ema_slow
    signal_line =
macd_line.ewm(span=signal,
adjust=False).mean()
    return macd_line, signal_line

def calculate_ema(close, period):
    return close.ewm(span=period,
adjust=False).mean()

def calculate_bollinger(close, period=20,
num_std=2):
    mid =
close.rolling(window=period).mean()
    std =
close.rolling(window=period).std()
    upper = mid + num_std * std
    lower = mid - num_std * std
    return upper, mid, lower

```

```

def calculate_stochastic(high, low, close,
period=14, smooth=3):
    lowest_low =
low.rolling(window=period).min()
    highest_high =
high.rolling(window=period).max()
    percent_k = 100 * (close - lowest_low)
/ (highest_high - lowest_low).replace(0,
1e-10)
    percent_d =
percent_k.rolling(window=smooth).mean()
    return percent_k, percent_d

```

```

# -----
-----
# 4) SIGNAL LOGIC (5-indicator scoring, per
timeframe)
# -----
-----

```

```

def score_timeframe(df, cfg):
    """
    Ek timeframe (df = high/low/close) ke
    liye 5 indicators check karke
    bullish aur bearish votes count karta
    hai.
    Returns: (direction, bullish_votes,
bearish_votes, last_price, details_dict)
    direction = "BUY" / "SELL" / "HOLD"
    """
    close, high, low = df["close"],
df["high"], df["low"]

```

```

    rsi = calculate_rsi(close,
cfg["RSI_PERIOD"])
    macd_line, macd_signal =
calculate_macd(close, cfg["MACD_FAST"],
cfg["MACD_SLOW"], cfg["MACD_SIGNAL"])
    ema_fast = calculate_ema(close,
cfg["EMA_FAST"])
    ema_slow = calculate_ema(close,
cfg["EMA_SLOW"])
    bb_upper, bb_mid, bb_lower =
calculate_bollinger(close,
cfg["BB_PERIOD"], cfg["BB_STD"])
    stoch_k, stoch_d =
calculate_stochastic(high, low, close,
cfg["STOCH_PERIOD"], cfg["STOCH_SMOOTH"])

    min_len = max(cfg["EMA_SLOW"],
cfg["BB_PERIOD"], cfg["MACD_SLOW"] +
cfg["MACD_SIGNAL"]) + 2
    if len(close) < min_len:
        return "HOLD", 0, 0, None, {}

    i, p = -1, -2 # latest and previous
index
    last_close = close.iloc[i]

    bullish = 0
    bearish = 0

    # 1) RSI
    if pd.notna(rsi.iloc[i]):
        if rsi.iloc[i] <
cfg["RSI_OVERSOLD"]:
            bullish += 1

```

```

        elif rsi.iloc[i] >
cfg["RSI_OVERBOUGHT"]:
        bearish += 1

# 2) MACD crossover
    if pd.notna(macd_line.iloc[i]) and
pd.notna(macd_signal.iloc[i]):
        if macd_line.iloc[p] <=
macd_signal.iloc[p] and macd_line.iloc[i] >
macd_signal.iloc[i]:
            bullish += 1
        elif macd_line.iloc[p] >=
macd_signal.iloc[p] and macd_line.iloc[i] <
macd_signal.iloc[i]:
            bearish += 1

# 3) EMA fast/slow crossover
    if pd.notna(ema_fast.iloc[i]) and
pd.notna(ema_slow.iloc[i]):
        if ema_fast.iloc[p] <=
ema_slow.iloc[p] and ema_fast.iloc[i] >
ema_slow.iloc[i]:
            bullish += 1
        elif ema_fast.iloc[p] >=
ema_slow.iloc[p] and ema_fast.iloc[i] <
ema_slow.iloc[i]:
            bearish += 1

# 4) Bollinger Bands (price band ke
bahar/kinare)
    if pd.notna(bb_lower.iloc[i]) and
pd.notna(bb_upper.iloc[i]):
        if last_close <= bb_lower.iloc[i]:
            bullish += 1

```

```

        elif last_close >=
bb_upper.iloc[i]:
            bearish += 1

    # 5) Stochastic Oscillator
    if pd.notna(stoch_k.iloc[i]) and
pd.notna(stoch_d.iloc[i]):
        oversold_zone = stoch_k.iloc[i] <
cfg["STOCH_OVERSOLD"]
        overbought_zone = stoch_k.iloc[i] >
cfg["STOCH_OVERBOUGHT"]
        cross_up = stoch_k.iloc[p] <=
stoch_d.iloc[p] and stoch_k.iloc[i] >
stoch_d.iloc[i]
        cross_down = stoch_k.iloc[p] >=
stoch_d.iloc[p] and stoch_k.iloc[i] <
stoch_d.iloc[i]
        if oversold_zone and cross_up:
            bullish += 1
        elif overbought_zone and
cross_down:
            bearish += 1

    details = {
        "rsi": round(rsi.iloc[i], 2) if
pd.notna(rsi.iloc[i]) else None,
        "macd": round(macd_line.iloc[i], 5)
if pd.notna(macd_line.iloc[i]) else None,
        "stoch_k": round(stoch_k.iloc[i],
2) if pd.notna(stoch_k.iloc[i]) else None,
    }

    if bullish >= cfg["MIN_CONFIRMATIONS"]
and bullish > bearish:

```

```

        return "BUY", bullish, bearish,
last_close, details
    elif bearish >=
cfg["MIN_CONFIRMATIONS"] and bearish >
bullish:
        return "SELL", bullish, bearish,
last_close, details
    else:
        return "HOLD", bullish, bearish,
last_close, details

def
generate_multi_timeframe_signal(candles_by_
tf, cfg):
    """
        candles_by_tf: dict jaisa {"1m": df,
"3m": df} (ya forex ke liye "1min"/"3min")
        Signal tabhi deta hai jab DONO
timeframes same direction confirm karein.
    """
    results = {}
    for tf, df in candles_by_tf.items():
        results[tf] = score_timeframe(df,
cfg)

    directions = [r[0] for r in
results.values()]
    if len(set(directions)) == 1 and
directions[0] != "HOLD":
        final_direction = directions[0]
    else:
        final_direction = "HOLD"

```

```

        return final_direction, results

# -----
# -----
# 5) TELEGRAM SENDING
# -----
# -----

def send_telegram_message(cfg, message,
                           chat_id=None):
    url =
    f"https://api.telegram.org/bot{cfg['TELEGRAM_BOT_TOKEN']}/sendMessage"
    payload = {
        "chat_id": chat_id if chat_id is
not None else cfg["TELEGRAM_CHAT_ID"],
        "text": message,
        "parse_mode": "Markdown",
    }
    try:
        resp = requests.post(url,
                              json=payload, timeout=15)
        if resp.status_code >= 300:
            print(f"[ERROR] Telegram send
failed: {resp.text}")
        else:
            print("[OK] Signal sent to
Telegram")
    except Exception as e:
        print(f"[ERROR] Could not send
Telegram message: {e}")

```

```

# -----
-----
# 6) STATE (avoid repeating the same signal
too often)
# -----
-----

def load_state(path):
    if os.path.exists(path):
        with open(path, "r") as f:
            return json.load(f)
    return {}

def save_state(path, state):
    with open(path, "w") as f:
        json.dump(state, f, indent=2)

# -----
-----
# 7) MAIN LOOP
# -----
-----

_forex_rotation_index = [0]

def check_market_once(cfg):
    state = load_state(cfg["STATE_FILE"])

    # ---- Crypto: har cycle mein SAB check
    hote hain (Kraken -> koi region-block nahi)
    ----

```

```

        for symbol in cfg["CRYPTO_SYMBOLS"]:
            try:
                candles_by_tf =
get_crypto_multi_timeframe(symbol, cfg)
                direction, results =
generate_multi_timeframe_signal(candles_by_
tf, cfg)
                handle_signal(cfg, state,
f"CRYPTO:{symbol}", direction, results)
            except Exception as e:
                print(f"[ERROR] Crypto
{symbol}: {e}")

        # ---- Forex: round-robin, har cycle
mein sirf EK pair check hota hai ----
        # (Twelve Data free plan ki 8 req/min
limit ke andar rehne ke liye)
        if cfg["FOREX_PAIRS"]:
            idx = _forex_rotation_index[0] %
len(cfg["FOREX_PAIRS"])
            pair = cfg["FOREX_PAIRS"][idx]
            _forex_rotation_index[0] += 1
            try:
                candles_by_tf = {
                    tf: get_forex_candles(pair,
tf, cfg["TWELVE_DATA_API_KEY"], cfg=cfg)
                    for tf in
cfg["FOREX_INTERVALS"]
                }
                direction, results =
generate_multi_timeframe_signal(candles_by_
tf, cfg)
                handle_signal(cfg, state,
f"FOREX:{pair}", direction, results)

```

```

        except Exception as e:
            print(f"[ERROR] Forex {pair}:
{e}")

        save_state(cfg["STATE_FILE"], state)

def handle_signal(cfg, state, key,
direction, results):
    if direction == "HOLD":
        return

    now = time.time()
    prev = state.get(key)
    if prev and prev.get("signal") ==
direction:
        if now - prev.get("time", 0) <
cfg["SIGNAL_COOLDOWN_SECONDS"]:
            return # abhi thodi der pehle
hi yehi signal bhej chuke hain

    tf_lines = []
    last_price = None
    for tf, (tf_dir, bulls, bears, price,
details) in results.items():
        last_price = price if price is not
None else last_price
        tf_lines.append(
            f" {tf}: {tf_dir} (bullish
{bulls}/5, bearish {bears}/5) "
            f"RSI={details.get('rsi')}
Stoch={details.get('stoch_k')}}"
        )

```

```

        message = (
            f"🤖 *{BOT_NAME}*\n"
            f"Pair: {key}\n"
            f"Signal: *{direction}*\n"
            f"Price: {last_price:.5f}\n"
            f"Timeframes:\n" +
            "\n".join(tf_lines) + "\n\n"
            f"⚠️ Yeh sirf technical signal hai,
financial advice nahi. Apni risk pe trade
karein."
        )
        print(message)
        send_telegram_message(cfg, message)
        state[key] = {"signal": direction,
"time": now}

def run_forever(cfg):
    print(f"{BOT_NAME} start ho gaya hai.
Har {cfg['CHECK_INTERVAL_SECONDS']} seconds
mein market check karega...")
    print("Telegram pe /signal bhej kar
turan best pair ka signal bhi maang sakte
hain.")

    import threading

    def background_scanner():
        while True:
            check_market_once(cfg)

    time.sleep(cfg["CHECK_INTERVAL_SECONDS"])

    scanner_thread =

```

```

threading.Thread(target=background_scanner,
daemon=True)
    scanner_thread.start()

    poll_telegram_commands(cfg)

# -----
# 8) ON-DEMAND /signal COMMAND (bot khud
# best pair chunta hai, turant deta hai)
# -----

def find_best_instant_signal(cfg):
    """
    Sab crypto pairs (fast, unlimited) + ek
    random forex pair scan karke
    jo sabse strong (zyada indicators
    agree) hai wo return karta hai.
    Designed to complete within a few
    seconds.
    """
    candidates = []

    for symbol in cfg["CRYPTO_SYMBOLS"]:
        try:
            candles_by_tf =
get_crypto_multi_timeframe(symbol, cfg)
            direction, results =
generate_multi_timeframe_signal(candles_by_
tf, cfg)
            total_votes = sum(r[1] + r[2]
for r in results.values())

```

```

        candidates.append((f"CRYPTO:
{symbol}", direction, results,
total_votes))
        except Exception as e:
            print(f"[ERROR] instant crypto
{symbol}: {e}")

    if cfg["FOREX_PAIRS"]:
        idx = _forex_rotation_index[0] %
len(cfg["FOREX_PAIRS"])
        pair = cfg["FOREX_PAIRS"][idx]
        _forex_rotation_index[0] += 1
        try:
            candles_by_tf = {
                tf: get_forex_candles(pair,
tf, cfg["TWELVE_DATA_API_KEY"], cfg=cfg)
                for tf in
cfg["FOREX_INTERVALS"]
            }
            direction, results =
generate_multi_timeframe_signal(candles_by_
tf, cfg)
            total_votes = sum(r[1] + r[2]
for r in results.values())
            candidates.append((f"FOREX:
{pair}", direction, results, total_votes))
        except Exception as e:
            print(f"[ERROR] instant forex
{pair}: {e}")

    if not candidates:
        return None

    non_hold = [c for c in candidates if

```

```

c[1] != "HOLD"]
    pool = non_hold if non_hold else
candidates
    return max(pool, key=lambda c: c[3])

def handle_instant_signal_request(cfg,
chat_id):
    start_time = time.time()
    best = find_best_instant_signal(cfg)

    if best is None:
        send_telegram_message(cfg, "⚠️ Abhi
data nahi mil saka, thodi der baad try
karein.", chat_id=chat_id)
        return

    key, direction, results, votes = best
    tf_lines = []
    last_price = None
    for tf, (tf_dir, bulls, bears, price,
details) in results.items():
        last_price = price if price is not
None else last_price
        tf_lines.append(
            f" {tf}: {tf_dir} (bullish
{bulls}/5, bearish {bears}/5) "
            f"RSI={details.get('rsi')}
Stoch={details.get('stoch_k')}}"
        )

    elapsed = round(time.time() -
start_time, 1)
    confidence_note = "✅ Strong

```

```
confirmation" if direction != "HOLD" else
"🕒 Koi strong signal nahi mila abhi, sabse
qareeb wala pair:"
```

```
    message = (
        f"🤖 *{BOT_NAME}* – On-Demand
Signal\n"
        f"Pair: {key}\n"
        f"Signal: *{direction}*\n"
        f"Price: {last_price:.5f}\n"
        f"{confidence_note}\n"
        f"Timeframes:\n" +
        "\n".join(tf_lines) + "\n\n"
        f"⌚ Generated in {elapsed}s\n"
        f"⚠️ Yeh sirf technical signal hai,
financial advice nahi. Apni risk pe trade
karein."
    )
    send_telegram_message(cfg, message,
chat_id=chat_id)
```

```
def normalize_pair_key(raw):
    return raw.upper().replace("/",
    "").replace("-", "").replace(" ", "")
```

```
def build_pair_lookup(cfg):
    crypto_lookup = {normalize_pair_key(s):
s for s in cfg["CRYPTO_SYMBOLS"]}
    forex_lookup = {normalize_pair_key(p):
p for p in cfg["FOREX_PAIRS"]}
    return crypto_lookup, forex_lookup
```

```

def normalize_timeframe(raw, market):
    raw = raw.lower().strip()
    digits = "".join(ch for ch in raw if
ch.isdigit()) or "1"
    if market == "crypto":
        return f"{digits}m"
    else:
        return f"{digits}min"

def handle_predict_request(cfg, chat_id,
args):
    if len(args) < 2:
        send_telegram_message(
            cfg,
            "Format: `/predict PAIR
TIMEFRAME`\n"
            "Misaal: `/predict EURUSD 1m`
ya `/predict BTCUSDT 3m`",
            chat_id=chat_id,
        )
        return

    raw_pair, raw_tf = args[0], args[1]
    crypto_lookup, forex_lookup =
build_pair_lookup(cfg)
    key = normalize_pair_key(raw_pair)

    market, actual_symbol = None, None
    if key in crypto_lookup:
        market, actual_symbol = "crypto",
crypto_lookup[key]
    elif key in forex_lookup:

```

```

        market, actual_symbol = "forex",
forex_lookup[key]
    else:
        send_telegram_message(
            cfg,
            f"⚠️ Pair `{raw_pair}` list
mein nahi mila. Available pairs:\n"
            f"Crypto: {'',
'.join(cfg['CRYPTO_SYMBOLS'])}\n"
            f"Forex: {'',
'.join(cfg['FOREX_PAIRS'])}",
            chat_id=chat_id,
        )
        return

    timeframe = normalize_timeframe(raw_tf,
market)

    try:
        if market == "crypto":
            candles_by_tf =
get_crypto_multi_timeframe(actual_symbol,
cfg)

            df =
candles_by_tf.get(timeframe,
candles_by_tf["1m"])
        else:
            df =
get_forex_candles(actual_symbol, timeframe,
cfg["TWELVE_DATA_API_KEY"], cfg=cfg)
    except Exception as e:
        send_telegram_message(cfg, f"⚠️
Data nahi mil saka: {e}", chat_id=chat_id)
    return

```

```

        direction, bullish, bearish, price,
details = score_timeframe(df, cfg)

        if direction == "BUY":
            call = "📈 UP (candle upar jaane ka
zyada imkaan)"
        elif direction == "SELL":
            call = "📉 DOWN (candle neeche
jaane ka zyada imkaan)"
        else:
            if bullish > bearish:
                call = "📈 Halka UP lean
(confirmation weak hai)"
            elif bearish > bullish:
                call = "📉 Halka DOWN lean
(confirmation weak hai)"
            else:
                call = "➡ Neutral / unclear
(koi clear direction nahi)"

        message = (
            f"🤖 *{BOT_NAME}* - Prediction\n"
            f"Pair: {actual_symbol}\n"
            f"Timeframe: {timeframe}\n"
            f"Current Price: {price:.5f}\n"
            f"Call: *{call}*\n"
            f"Bullish votes: {bullish}/5 |
Bearish votes: {bearish}/5\n"
            f"RSI: {details.get('rsi')} | MACD:
{details.get('macd')} | Stoch:
{details.get('stoch_k')}\n\n"
            f"⚠ Ye ek probability-based
estimate hai, guarantee nahi. Har candle ka

```

```

result "
    f"random bhi ho sakta hai – apni
risk pe faisla karein."
)
    send_telegram_message(cfg, message,
chat_id=chat_id)

def poll_telegram_commands(cfg):
    """Telegram se /signal command sunta
hai aur turant response deta hai."""
    offset = None
    print("Telegram command listener chalu
ho gaya (/signal bhejein).")
    while True:
        try:
            url =
f"https://api.telegram.org/bot{cfg['TELEGRAM_BOT_TOKEN']}/getUpdates"
            params = {"timeout": 30}
            if offset is not None:
                params["offset"] = offset
            resp = requests.get(url,
params=params, timeout=35)
            data = resp.json()
            for update in
data.get("result", []):
                offset =
update["update_id"] + 1
                message =
update.get("message", {})
                text = (message.get("text")
or "").strip()
                text_lower = text.lower()

```

```

        chat_id =
message.get("chat", {}).get("id")
        if
text_lower.startswith("/signal") and
chat_id is not None:

handle_instant_signal_request(cfg, chat_id)
        elif
text_lower.startswith("/predict") and
chat_id is not None:
            args = text.split()[1:]

handle_predict_request(cfg, chat_id, args)
        elif
text_lower.startswith("/start") and chat_id
is not None:

            send_telegram_message(
                cfg,
                f"👋 *{BOT_NAME}*
ready hai!\n\n"
                f"`/signal` – bot
khud best pair chun kar turant signal
dega\n"
                f"`/predict PAIR
TIMEFRAME` – ek specific pair/timeframe ka
UP/DOWN call\n"
                f"Misaal: `/predict
EURUSD 1m` ya `/predict BTCUSDT 3m`,
                chat_id=chat_id,
            )
        except Exception as e:
            print(f"[ERROR] Telegram
polling: {e}")
            time.sleep(3)

```

```
if __name__ == "__main__":  
    run_forever(CONFIG)
```